CMSC132: Abstraction and Encapsulation

Chapter 4

By Naren Ayinala

May 2026

Contents

1 Abstraction: Show What Matters, Hide How **2**

1.1 Abstraction with Interfaces 2

2 Encapsulation: Protect an Object's State **2**

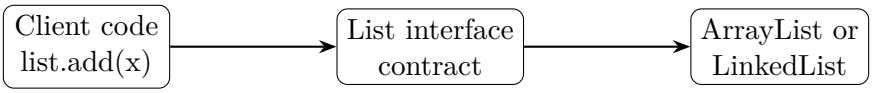
2.1 Why Returning Internal Mutable Objects Is Dangerous 3

3 Abstraction vs. Encapsulation **3**

3.1 Practice 3

1 Abstraction: Show What Matters, Hide How

Abstraction means exposing a useful model or contract while hiding implementation details. For example, when you call `list.add("A")`, you care that the list gains an element; you do not need to know whether the list shifts array elements or relinks nodes. The public methods describe *what* clients can do, not necessarily *how* it is done.



1.1 Abstraction with Interfaces

```
interface Storage {
    void save(String key, String value);
    String load(String key);
}
class MemoryStorage implements Storage {
    private final java.util.Map<String,String> data =
        new java.util.HashMap<>();
    public void save(String k, String v) { data.put(k, v); }
    public String load(String k) { return data.get(k); }
}
Storage s = new MemoryStorage();
s.save("course", "CMSC132");
System.out.println(s.load("course"));
```

Code written against the `Storage` interface can switch to a file-backed implementation without rewriting the caller. This is one of the practical benefits of abstraction.

2 Encapsulation: Protect an Object's State

Encapsulation groups data and the operations on that data inside a class and controls access to internal state. It is not merely "making fields private": the goal is to preserve class invariants, meaning conditions that must always remain true. A bank account should not have a negative balance if overdrafts are forbidden.

```
class BankAccount {
    private int balanceCents;
    BankAccount(int opening) {
        if (opening < 0) throw new IllegalArgumentException();
        balanceCents = opening;
    }
    public int getBalanceCents() { return balanceCents; }
    public void deposit(int cents) {
        if (cents <= 0) throw new IllegalArgumentException();
        balanceCents = Math.addExact(balanceCents, cents);
    }
    public boolean withdraw(int cents) {
        if (cents <= 0) throw new IllegalArgumentException();
        if (cents > balanceCents) return false;
        balanceCents -= cents;
        return true;
    }
```

```
}
```

2.1 Why Returning Internal Mutable Objects Is Dangerous

```
class Course {
    private final java.util.List<String> students =
        new java.util.ArrayList<>();
    public void enroll(String name) { students.add(name); }
    public java.util.List<String> students() {
        return java.util.List.copyOf(students); // defensive snapshot
    }
}
```

If this method returned the actual mutable `students` list, a caller could modify it without passing through `enroll`. Defensive copies or unmodifiable views protect the internal representation. Note that a copy of a collection may still share mutable element objects.

3 Abstraction vs. Encapsulation

Question	Abstraction	Encapsulation
Main goal	Hide complexity	Protect state
Emphasis	What operations exist	Who can change data
Tools	Interfaces, ADTs	private, validation

These concepts work together. An interface abstracts a collection's operations; a concrete class encapsulates the array or nodes that implement those operations. **Common exam trap:** A public getter and setter for every field does not automatically produce good encapsulation: a setter can expose invalid state transitions.

3.1 Practice

1. A class hides an array but exposes `add` and `remove`. Which principle? Both abstraction (public operations) and encapsulation (hidden representation). **2.** Why validate in setters or methods? To preserve invariants. **3.** Can a class be well encapsulated without implementing an interface? Yes.